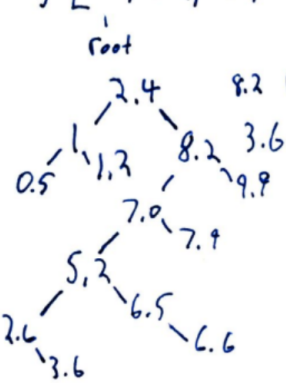
Elijah k

assignment 6

Made with Goodnotes

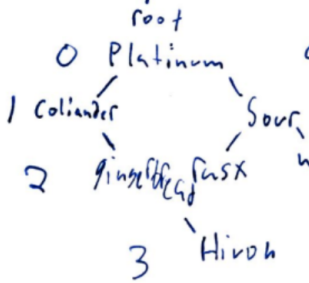
1) a) [2.4, 8.2, 7.0, 5.2, 1.0, 6.5, 2.6, 3.6, 7.9, 0.5, 6.6, 4.9, 1.2]



8.2 r, 7.0 rL, 5.2 rLL, 1.0 L, 6.5 rLLr, 2.6 rLLL, 3.6 rLLLr, 7.9 rLr, 0.5 LL, 6.6 rLLrr, 4.9 rr, 1.2 Lr

height 5 → 2.4 → 8.2 → 7 → 5.2 → 2.6 → 3.6

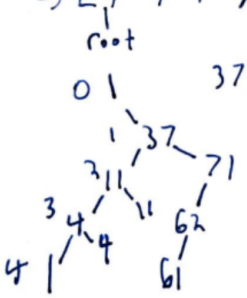
B) ["Platinum", "Coliander", "Sour", "willow", "Tasx", "gingerbread", "Hivon"]



Coliander L, Sour R, willow rr, Tasx rL, gingerbread Rr, Hivon Lrr

height: 3

C) [1, 37, 11, 71, 62, 4, 61, 1, 11, 4]



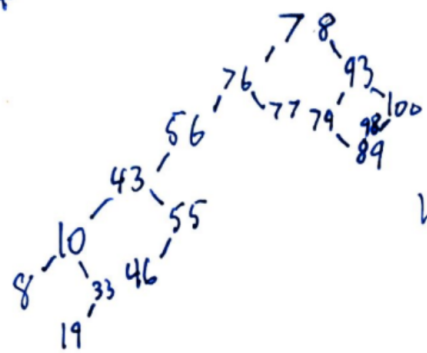
37r, 11rL, 71rr, 62rrL, 4rLL, 61rrLL, 1rLLL, 11rLr, 4rLLr

height: 4

Made with Goodnotes

D) [78, 76, 56, 43, 55, 77, 10, 93, 46, 79, 33, 8, 89, 100, 98, 19]

root



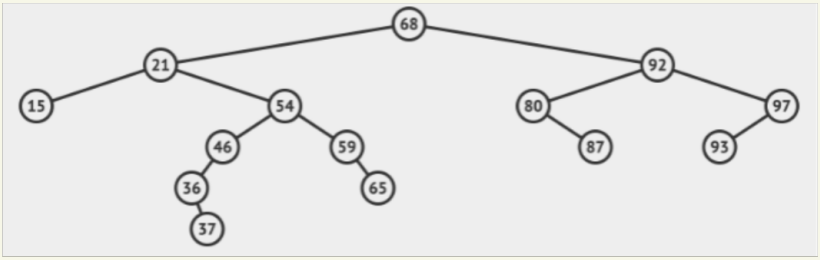
76L, 56LL, 43LLL, 55LLLR, 77LR,
10LLLL, 93R, 46LLLRL, 79RL,
33LLLLR, 8LLLLL, 89RLR, 100RR, 98RRE,
19LLLLRL

height: 6 [78, 76, 56, 43, 10, 33, 19]

Made with Goodnotes

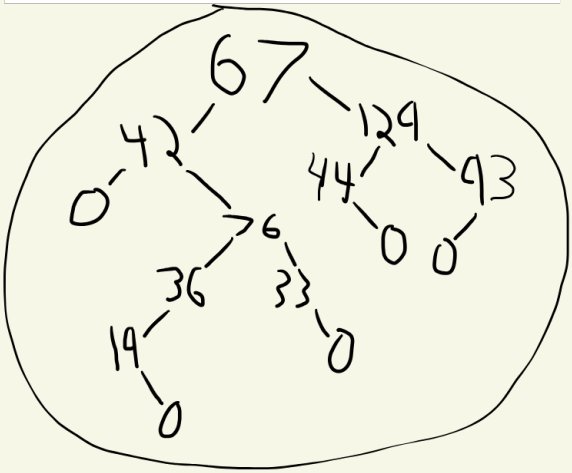
2)

a)



Pre order - 68, 21, 15, 54, 46, 36, 37, 59, 65, 92, 80, 87, 97, 93

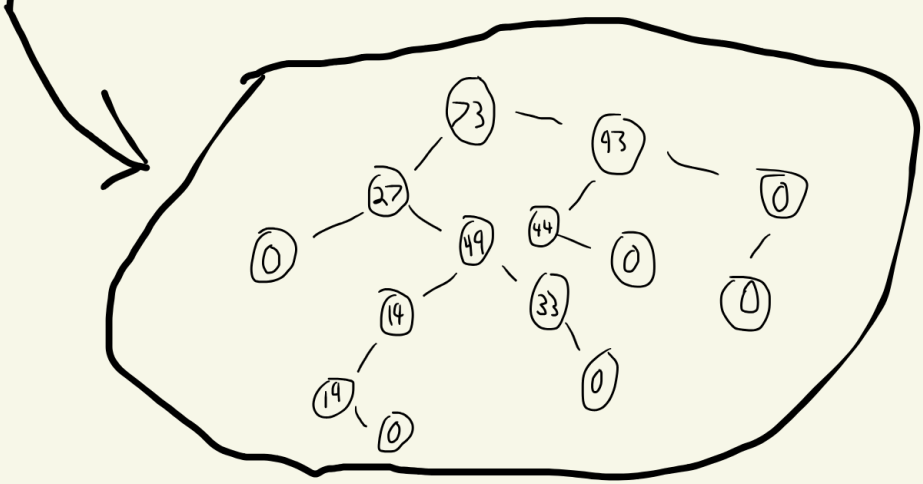
node.data = left.data + (right.data/2)



In order - 15, 21, 36, 37, 46, 54, 59, 65, 68, 80, 87, 92, 93, 97

node.data = left.data + (right.data/2)

Made with Goodnotes



b. **[2 points]** Are the resulting trees BSTs?

Preorder: No it is not a BST, the new root is 67 and the left child is 42 but the right child of 42 is 76 which means there is a number greater than 67 in its left subtree which isnt allowed.

Inorder: No it is not a BST, if you go to node 27 then its right child 49. 49's left child is 19 which is smaller than 27 so it shouldnt be in that subtree.

c. **[2 points]** Are the resulting trees AVLs?

No since neither are BST they cannot be AVLs since that is a requirement.

7 (text) Algorithm Analysis [20 points]
For each of the code you wrote for problems 3-6, explain its time complexity and space complexity for each problem using Big-O and Big-Ω notation. Explain how you arrived at your answer.

Problem 3: The time complexity is O(N) Where N is the length of the input text because we must look at four steps. Counting the frequencies takes O(N), building the tree takes O(K^2) time where K is the number of characters, creating the codes takes O(K) and building the encoded string takes O(N). Since K is the alphabet which is a fixed number or constant, O(N) ends up being the dominant factor making time complexity O(N).
Space Complexity is O(N) where N is the length of the input text becuase my algorithim has to make enough memory for the frequency map which is O(K) where K is the length of the alphabet, the tree nodes O(K), and the encoded output string which is O(N). K is a constant again which means that O(N) is the dominating which makes space complexity O(N) as N grows.

Made with Goodnotes

Problem 4: Time complexity is O(N) where N is the size of the array because this tree is not a normal BST which means that we cannot search this in O(log N). In the worst case scenario the number we are looking for could be at the end of our array which would mean that the for loop would execute exactly N times on each and every Node in the tree.
Space Complexity is O(N) where N is the size of the array because the first thing that we do is "int[] unencrypted = new int[bt.length]" this means that everytime we run this program we create room in the memory that is grows at the same rate of our input size. This makes it O(N) .

Problem 5: The time complexity is O(N) where N is the number of strings in the array because no matter what everytime we run the program it will loop through each of the numbers in the array or in the heap since we have the for loop.
Space complexity is O(N) where N is the number of strings in the array because in the worst case scenario someone could input 100,000 for x which would mean that every word in the list would be added to the arraylist that create which would mean that as our input size grows so does our memory.

Problem 6: The time complexity is O(N^2) where N is the total number of nodes in the tree because in the program we traverse and also delete. When we traverse we visist each and every single node which is N. Whenever we delete something it is O(H) time per node where H is the tree's height. This is becuase in the worst case scenario the computer would have to walk down the whole tree. If the tree was like a linked list it would have to delete N nodes and each deletion would be N steps is makes it N x H but we can make that O(N^2).
Space complexity is O(N) where N is the total number of nodes in the tree becuase in the worst case scenario the arraylist would have to store all nodes if they were all in the range which is N. The recurssion stack will also be N levels deep but N + N ends up just being O(N).

Made with Goodnotes